EH/Unwind Analyzer

Актуализация документа для релиза 1.0.0

Документ: EH_Unwind_Analyzer_Полное_руководство_по_признакам_и_эвристикам.pdf

Дата финальной проверки: 19 июня 2026 года

Этот PDF сохранен как теоретический и презентационный материал проекта.
Актуальный статус релиза подтвержден отдельными evidence-документами:

- MinGW-w64 Release CTest: 16/16 PASS
- MSVC Release CTest: 16/16 PASS
- Real benign static corpus: 106/106 valid, non-zero scores: 0
- Labeled static corpus: 14/14 valid, 12/12 anomalous samples flagged
- Dynamic smoke: 4/4 PASS
- Full Windows dynamic corpus: 124/124 valid reports, unexpected failures: 0

Актуальные release-файлы: release/RELEASE_STATUS_1.0.0_RU.md, release/VERIFIED_WINDOWS_BUILD.json и AUDIT_INVESTOR_READINESS_RU.md.

Оригинальное содержание документа начинается со следующей страницы.

EH/UNWIND ANALYZER

Полное руководство по признакам, нарушениям и эвристикам

От формальных повреждений PE и RUNTIME_FUNCTION до placement, reuse, динамических сигналов, score, confidence и ручной проверки

Документ объясняет каждый реализованный признак простым языком, но без потери технической точности. Главная цель — понимать, что именно обнаружила программа, почему это важно и какой вывод допустимо сделать.

Реализация: 1.0.0-rc1
Согласовано с исходным кодом на 9 июня 2026 года

1. Главное различие: нарушение и эвристика

Формальное структурное нарушение

Правило напрямую следует из внутренней модели формата: диапазон должен существовать, End должен быть больше Begin, ссылка должна попадать в image, обязательные bytes должны присутствовать. Такое finding надёжно говорит о несогласованности структуры, но всё ещё не доказывает злой умысел.

Контекстная эвристика

Структура может быть технически читаемой, но расположение или статистический паттерн отличается от обычного compiler output. Примеры: metadata в необычной секции, общий handler, сильный reuse. Эвристика нужна для triage и всегда допускает нормальное объяснение.

Комбинированный сигнал

Несколько разных признаков подтверждают одну гипотезу. Например, metadata одновременно writable и связана с writable handler. Комбинация повышает приоритет, потому что её сложнее объяснить одной случайной особенностью.

Что означает «обнаружена аномалия»

- Программа нашла отличие от формального или ожидаемого инженерного правила.
- Аномалия не равна вредоносности, атаке, упаковщику или конкретному protector.
- Сильное нарушение говорит о недостоверности metadata; слабая эвристика только определяет направление ручного анализа.
- Итог следует делать по совокупности findings, evidence families, mitigations и runtime evidence.

2. Как данные превращаются в вывод



Пять evidence families

Семейство	Что означает	Сила
`table_integrity`	Повреждения секций и таблицы диапазонов функций.	Обычно самая высокая.
`unwind_integrity`	Невозможность или противоречивость декодирования UNWIND_INFO.	Высокая.
`metadata_placement`	Роль metadata/handler не соответствует секции или правам.	От средней до высокой.
`unwind_reuse`	Статистически необычное совместное использование одной записи.	Контекстная.
`handler_dominance`	Почти все handlers сходятся к одному target.	Контекстная.

Почему несколько families важнее нескольких одинаковых findings

Если parser видит unusual section, writable section и handler в той же необычной схеме, это может быть одна причина. Если одновременно нарушены диапазоны RUNTIME_FUNCTION и структура UNWIND_INFO, появляются независимые подтверждения из разных уровней формата.

3. Проверки, которые выполняются ещё до эвристик

Некоторые ошибки настолько фундаментальны, что PE parser прекращает работу до формирования heuristic score. Это не означает, что они менее важны: наоборот, файл не проходит минимальный входной контракт.

Проверка	Что означает ошибка
MZ и PE signature	Файл не имеет ожидаемых DOS/NT signatures.
e_lfanew и headers bounds	Заголовки находятся за пределами файла.
Machine = AMD64	Архитектура не поддерживается текущей моделью.
OptionalHeader = PE32+	Это не 64-разрядный PE32+ image.
SizeOfImage > 0	Виртуальный размер образа не определён.
SizeOfHeaders	Заголовки имеют нулевой размер или выходят за файл.
File/SectionAlignment	Объявлены нулевые параметры выравнивания.
DataDirectory count	Optional header не содержит Exception Directory entry.
Section headers	Таблица секций обрезана.
Exception table size	RUNTIME_FUNCTION table должна быть кратна 12 bytes и отображаться в файл.

Почему эти ошибки не получают score

Score нужен для сравнения findings внутри успешно разобранной поддерживаемой модели. Если базовый PE contract не выполнен, корректнее вернуть явную parse error, а не делать вид, что результат можно сравнивать с обычным образом.

4. Приоритетная карта всех 27 findings

Приоритет	Что входит	Как читать
A	OOB, invalid ranges, overlaps, parse failures, внутренние UNWIND_INFO anomalies.	Сначала проверять целостность структуры.
B	Сильное placement, raw overlap, unsorted/duplicates и combined corroboration.	Проверять вместе с section map и disassembly.
C	Reuse, dominance, unusual name, misalignment, empty runtime table.	Контекстные признаки; искать нормальное объяснение.

Важно: эта приоритетная группировка объясняет порядок ручного анализа. Она не заменяет реальные поля severity и weight, указанные в каждой карточке.

5. Приоритет A: формальные повреждения и сильные integrity-проблемы

Эти findings проверяются первыми. Они показывают, что адреса, диапазоны или структура metadata внутренне несогласованы.

pe_section_raw_oob: секция выходит за пределы файла

ПРИОРИТЕТ A · сильное структурное нарушение | family: table_integrity | severity: high | weight: +25

Простыми словами	В заголовке секции написано, что её байты находятся там, где файл уже закончился.
Что проверяет код	Для секции с RawSize > 0 вычисляется RawPointer + RawSize. Если конец больше фактического размера файла, увеличивается счётчик raw_oob.
Зачем это нужно	Не позволить parser читать отсутствующие байты и обнаружить противоречие между заголовком PE и реальным содержимым.
Почему это важно	Это сильный признак повреждённого, обрезанного или намеренно запутанного файла. Данные секции физически недоступны в заявленном диапазоне.
Нормальное объяснение	Практически полезных нормальных объяснений мало. Возможен повреждённый download, некорректный упаковщик или нестандартный генератор.
Как проверить вручную	Сравнить PointerToRawData, SizeOfRawData и размер файла в PE viewer или hex editor. Проверить, действительно ли требуемые байты отсутствуют.
Как трактовать	Сильное структурное нарушение. Оно говорит о некорректности PE-layout, но само по себе не доказывает malicious intent.

pe_exception_directory_oob: каталог исключений неполон или выходит за SizeOfImage

ПРИОРИТЕТ A · сильное структурное нарушение | family: table_integrity | severity: high | weight: +25

Простыми словами	Главный указатель на таблицу x64 exception metadata ведёт в недопустимый диапазон.
Что проверяет код	Если RVA и Size exception directory не являются одновременно нулевыми или ненулевыми либо RVA + Size превышает SizeOfImage.
Зачем это нужно	Проверить, что вход в систему RUNTIME_FUNCTION вообще указывает на существующую часть виртуального образа.
Почему это важно	Без корректного directory невозможно надёжно прочитать таблицу функций. Это базовое противоречие PE optional header.
Нормальное объяснение	Ресурсный образ может не иметь каталога, но тогда оба поля должны быть нулевыми. Неполная пара полей нормой не является.
Как проверить вручную	Открыть Data Directory[Exception], пересчитать RVA + Size, сравнить с SizeOfImage и секциями.
Как трактовать	Один из самых важных findings: он ставит под сомнение целостность всей EH/Unwind-области.

rf_overlaps: диапазоны функций пересекаются

ПРИОРИТЕТ A · сильное структурное нарушение | family: table_integrity | severity: high | weight: +25

Простыми словами	Один адрес машинного кода одновременно попадает в две или больше записей RUNTIME_FUNCTION.
Что проверяет код	Записи сортируются по BeginAddress. Если начало очередной записи меньше максимального уже встреченного EndAddress, фиксируется overlap.
Зачем это нужно	Сохранить однозначное соответствие адреса исполнения одной записи unwind metadata.
Почему это важно	ОС или анализатор может получить неоднозначный ответ, какую запись использовать для раскрутки конкретного адреса.
Нормальное объяснение	Для обычной compiler-generated таблицы пересечения не ожидаются. Возможны ручные, защитные или повреждённые layouts.
Как проверить вручную	Отсортировать .pdata по BeginAddress и выписать конфликтующие [Begin, End) диапазоны. Сопоставить их с функциями в Ghidra.
Как трактовать	Сильное структурное нарушение таблицы, приоритетное для ручного анализа.

rf_oob: BeginAddress или EndAddress выходит за SizeOfImage

ПРИОРИТЕТ A · сильное структурное нарушение | family: table_integrity | severity: medium при ≤1%, high при >1% | weight: +10 / +25

Простыми словами	Запись утверждает, что функция находится за пределами загруженного образа.
Что проверяет код	BeginAddress >= SizeOfImage или EndAddress > SizeOfImage. Массовость выше 1% усиливает finding.
Зачем это нужно	Проверить, что описываемый код физически может принадлежать этому PE image.
Почему это важно	Такой диапазон нельзя корректно сопоставить с кодом текущего образа; возможны повреждение, подмена или ошибочная таблица.
Нормальное объяснение	Единичный дефект может появиться из-за повреждения файла или некачественного генератора. Это всё равно требует проверки.
Как проверить вручную	Сравнить Begin/End с SizeOfImage, найти соответствующие секции и убедиться, что диапазон не является арифметической ошибкой.
Как трактовать	Сильнее при массовом характере. Порог 1% отличает единичную запись от системной проблемы.

unwind_rva_oob: UnwindInfoRVA отсутствует или выходит за SizeOfImage

ПРИОРИТЕТ A · сильное структурное нарушение | family: table_integrity | severity: medium при ≤1%, high при >1% | weight: +10 / +25

Простыми словами	Запись функции ссылается на описание раскрутки, которого нет в допустимом адресном пространстве образа.
Что проверяет код	UnwindInfoRVA равен нулю либо не меньше SizeOfImage. Массовость выше 1% повышает тяжесть.
Зачем это нужно	Убедиться, что каждая nonleaf-запись ведёт к потенциально существующей UNWIND_INFO.
Почему это важно	Раскрутка такой функции не может быть выполнена по указанной ссылке.
Нормальное объяснение	Нулевая или внешняя ссылка для обычной RUNTIME_FUNCTION не является ожидаемым состоянием.
Как проверить вручную	Перейти по UnwindData в hex/PE viewer, проверить RVA mapping и наличие минимум четырёх байтов заголовка.
Как трактовать	Формальная проблема ссылки. Если затронута большая доля функций, это системное повреждение.

unwind_parse_fail: UNWIND_INFO невозможно безопасно разобрать

ПРИОРИТЕТ A · сильное структурное нарушение | family: unwind_integrity | severity: medium при ≤1%, high при >1% | weight: +10 / +25

Простыми словами	Parser дошёл до записи, но её структура обрывается или противоречит формату настолько, что продолжать небезопасно.
Что проверяет код	unwind_parse возвращает false: недоступен header, неизвестная version, массив кодов/operand/tail обрезан или chain target невозможно прочитать.
Зачем это нужно	Не угадывать структуру при нехватке байтов и не создавать ложную интерпретацию повреждённых данных.
Почему это важно	Часть функций остаётся без достоверного описания раскрутки. Массовые failures указывают на системную проблему.
Нормальное объяснение	Возможен неподдерживаемый новый формат, например будущий V3, либо обрезанный файл. Это не всегда атака.
Как проверить вручную	Изучить parse_error конкретной функции, открыть blob по RVA, сверить Version, CountOfCodes, размер operands и tail.
Как трактовать	Failure важнее обычного anomaly bit: структура не была полностью декодирована.

pe_section_virtual_oob: виртуальный диапазон секции выходит за SizeOfImage

ПРИОРИТЕТ A · сильное структурное нарушение | family: table_integrity | severity: high | weight: +20

Простыми словами	Секция по заголовку должна занимать память за пределами всего PE image.
Что проверяет код	VirtualAddress + max(VirtualSize, RawSize) превышает SizeOfImage.
Зачем это нужно	Проверить внутреннюю согласованность виртуальной геометрии, которую должен использовать loader.
Почему это важно	RVA mapping становится противоречивым: секция обещает адреса, не входящие в заявленный образ.
Нормальное объяснение	Обычно это ошибка генерации, повреждение или intentional malformed layout.
Как проверить вручную	Пересчитать virtual end каждой секции и сравнить с SizeOfImage и SectionAlignment.
Как трактовать	Сильный image-layout signal, но вывод относится к структуре, а не к назначению программы.

pe_section_virtual_overlap: виртуальные диапазоны секций пересекаются

ПРИОРИТЕТ A · сильное структурное нарушение | family: table_integrity | severity: high | weight: +20

Простыми словами	Один RVA одновременно принадлежит нескольким секциям.
Что проверяет код	Диапазоны [VirtualAddress, VirtualAddress + max(VirtualSize, RawSize)) двух секций пересекаются.
Зачем это нужно	Обеспечить однозначное определение секции по RVA, прав доступа и mapping в файл.
Почему это важно	Одинаковый адрес получает разные имена, права и файловые источники; это ломает доверие к дальнейшему анализу.
Нормальное объяснение	Нетипично для нормального PE image. Встречается в malformed или anti-analysis layouts.
Как проверить вручную	Построить карту RVA всех секций, сравнить пересечения и проверить, как файл воспринимают loader и PE tools.
Как трактовать	Сильное структурное противоречие; особенно важно для placement findings.

invalid_ranges: EndAddress не больше BeginAddress

ПРИОРИТЕТ A · сильное структурное нарушение | family: table_integrity | severity: medium при ≤1%, high при >1% | weight: +10 / +20

Простыми словами	У записи функции нулевая или отрицательная в логическом смысле длина.
Что проверяет код	Для RUNTIME_FUNCTION выполняется EndAddress <= BeginAddress.
Зачем это нужно	Проверить базовый инвариант полуоткрытого диапазона [Begin, End).
Почему это важно	Запись не может описывать непустой участок машинного кода и нарушает модель поиска функции.
Нормальное объяснение	Практически всегда дефект metadata. Единичный случай может быть следствием порчи одного поля.
Как проверить вручную	Выписать Begin/End, вычислить размер и найти соответствующие байты/функцию в дизассемблере.
Как трактовать	Формальное нарушение. При доле выше 1% считается широкой проблемой таблицы.

unwind_anomalies: UNWIND_INFO разобрана, но содержит внутренние противоречия

ПРИОРИТЕТ A · сильное структурное нарушение | family: unwind_integrity | severity: medium при ≤5%, high при >5% | weight: +10 / +20

Простыми словами	Байтов достаточно для чтения, однако version/opcode/operand/chain/alignment не соответствуют ожидаемой логике.
Что проверяет код	После успешного parse поле UnwindSummary.anomalies не равно нулю. Конкретные 12 битов подробно разобраны далее.
Зачем это нужно	Отделить полностью нечитаемые записи от читаемых, но подозрительных или несогласованных.
Почему это важно	ОС может иначе интерпретировать запись, а аналитический вывод о prolog/stack может оказаться неверным.
Нормальное объяснение	Возможны редкие toolchain encodings, будущие расширения или консервативная sanity-проверка operand.
Как проверить вручную	Посмотреть anomaly mask, декодированные operations и сравнить их с реальным прологом функции.
Как трактовать	Агрегированный finding. Для серьёзного вывода нужно назвать конкретный UA_* bit.

6. Приоритет B: размещение и подтверждающие комбинации

Эти findings часто указывают на сильное противоречие роли данных, прав секции или сочетание нескольких сигналов.

pe_section_raw_overlap: файловые диапазоны секций пересекаются

ПРИОРИТЕТ B · сильный или подтверждающий сигнал | family: table_integrity | severity: medium | weight: +15

Простыми словами	Одни и те же байты файла заявлены как содержимое нескольких секций.
Что проверяет код	Диапазоны PointerToRawData + SizeOfRawData двух секций пересекаются.
Зачем это нужно	Обнаружить неоднозначное владение байтами и возможное сокрытие данных за конфликтующими section headers.
Почему это важно	Разные инструменты могут по-разному показывать или интерпретировать одни байты.
Нормальное объяснение	Некоторые нестандартные упаковщики могут использовать shared raw data, но это требует ручной проверки.
Как проверить вручную	Сравнить raw ranges в hex editor и определить, одинаковое ли содержимое отображают конфликтующие секции.
Как трактовать	Средне-сильный structural signal; virtual overlap считается ещё серьёзнее.

rf_unsorted: таблица не отсортирована по BeginAddress

ПРИОРИТЕТ B · сильный или подтверждающий сигнал | family: table_integrity | severity: medium | weight: +15

Простыми словами	Следующая запись начинается раньше предыдущей.
Что проверяет код	При чтении исходного порядка BeginAddress текущей записи меньше предыдущего BeginAddress.
Зачем это нужно	Проверить ожидаемую организацию exception table для поиска функции по адресу.
Почему это важно	Нарушается предположение об упорядоченной таблице, что может мешать lookup и указывать на ручную модификацию.
Нормальное объяснение	Возможен нестандартный генератор, но для обычного compiler/linker output не ожидается.
Как проверить вручную	Сохранить исходный порядок .pdata, отсортировать копию и сравнить инверсии.
Как трактовать	Сам по себе слабее overlap/OOB, но усиливает другие table-integrity findings.

duplicate_begin: несколько записей имеют одинаковый BeginAddress

ПРИОРИТЕТ B · сильный или подтверждающий сигнал | family: table_integrity | severity: medium | weight: +10

Простыми словами	Для одного начала функции существуют несколько описаний.
Что проверяет код	После сортировки соседние записи имеют одинаковое BeginAddress.
Зачем это нужно	Выявить неоднозначное соответствие одной точки входа нескольким диапазонам/unwind records.
Почему это важно	Записи могут конфликтовать по EndAddress или UnwindInfoRVA.
Нормальное объяснение	Редкий linker/toolchain artifact возможен, но должен быть объяснён содержанием записей.
Как проверить вручную	Сравнить все записи с одинаковым BeginAddress и определить, идентичны ли остальные поля.
Как трактовать	Средний сигнал. Становится существенно сильнее вместе с overlaps или разными unwind targets.

unwind_in_exec_section: UNWIND_INFO находится в executable section

ПРИОРИТЕТ B · сильный или подтверждающий сигнал | family: metadata_placement | severity: high | weight: +20

Простыми словами	Данные раскрутки размещены в области, которую процессу разрешено исполнять как код.
Что проверяет код	Секция, содержащая UnwindInfoRVA, имеет IMAGE_SCN_MEM_EXECUTE.
Зачем это нужно	Проверить соответствие роли данных и прав памяти. Обычные metadata чаще находятся в non-executable .xdata/.rdata.
Почему это важно	Совмещение кода и metadata нетипично и может быть связано с ручной упаковкой, interception или redirected layout.
Нормальное объяснение	Монолитная секция, нестандартный linker script или защитное ПО могут сознательно смешивать данные и код.
Как проверить вручную	Проверить section permissions, соседние bytes, references и наличие реальных инструкций по этому адресу.
Как трактовать	Сильная placement-эвристика, но не формальное доказательство повреждения.

unwind_writable_section: UNWIND_INFO находится в writable section

ПРИОРИТЕТ B · сильный или подтверждающий сигнал | family: metadata_placement | severity: medium при ≤5%, high при >5% | weight: +10 / +18

Простыми словами	Процесс может изменять статические unwind-данные во время работы.
Что проверяет код	Секция UnwindInfoRVA имеет IMAGE_SCN_MEM_WRITE. Если затронуто больше n/20 записей, finding усиливается.
Зачем это нужно	Выделить metadata, чья неизменность не защищена section permissions.
Почему это важно	Динамическая подмена unwind может изменить обработку исключений или затруднить анализ.
Нормальное объяснение	JIT/runtime scenario, loader-generated данные или нестандартная секция могут быть writable по законной причине.
Как проверить вручную	Проверить, кто записывает в область, наблюдать memory protection и сравнить bytes до/после запуска.
Как трактовать	Требует manual review. Особенно силён вместе с writable handler target.

handler_nonexec_section: handler RVA находится вне executable section

ПРИОРИТЕТ B · сильный или подтверждающий сигнал | family: metadata_placement | severity: high | weight: +15

Простыми словами	Адрес, который должен вести на код обработчика, указывает в неисполняемые данные или вообще ни в одну секцию.
Что проверяет код	Для has_handler секция не найдена либо не имеет IMAGE_SCN_MEM_EXECUTE.
Зачем это нужно	Проверить соответствие роли адреса обработчика и возможности процессора исполнять его.
Почему это важно	Это прямое противоречие ожидаемой роли target. Возможны broken pointer, thunk outside model или intentional redirection.
Нормальное объяснение	Mapping может быть необычным, а реальный код появляться динамически, но статическая ссылка всё равно требует объяснения.
Как проверить вручную	Перейти по HandlerRVA, проверить section rights, disassembly, relocations и runtime memory map.
Как трактовать	Сильная placement-проблема и один из ключевых признаков для static/dynamic correlation.

handler_writable_section: handler находится в executable+writable section

ПРИОРИТЕТ B · сильный или подтверждающий сигнал | family: metadata_placement | severity: medium | weight: +10

Простыми словами	Код обработчика можно изменять во время исполнения.
Что проверяет код	Секция HandlerRVA существует и имеет IMAGE_SCN_MEM_WRITE; executable проверяется отдельным finding.
Зачем это нужно	Выделить mutable handler code, который может быть сгенерирован или подменён динамически.
Почему это важно	Writable executable memory повышает возможность runtime modification и усложняет доверие к статическому образу.
Нормальное объяснение	JIT, hot patching, runtime-generated stubs или особый loader могут использовать RWX.
Как проверить вручную	Проверить исходные section flags, VirtualProtect changes и фактические bytes handler во время запуска.
Как трактовать	Контекстно сильный сигнал, особенно с writable metadata или runtime function-table calls.

placement_exec_nonexec_handler_combo: metadata исполняема, а handler неисполняем

ПРИОРИТЕТ B · сильный или подтверждающий сигнал | family: metadata_placement | severity: high | weight: +10 дополнительно

Простыми словами	Роли областей выглядят перевёрнутыми: данные разрешено исполнять, а предполагаемый код обработчика — нет.
Что проверяет код	Одновременно unwind_exec_section > 0 и handler_nonexec_section > 0.
Зачем это нужно	Поднять приоритет взаимно согласованного противоречия размещения.
Почему это важно	Такую комбинацию сложнее объяснить одним случайным необычным section name.
Нормальное объяснение	Возможна сложная runtime mapping схема, но она должна быть подтверждена динамически.
Как проверить вручную	Сопоставить обе секции, посмотреть переход управления и реальную memory protection после загрузки.
Как трактовать	Не самостоятельное новое нарушение, а усилитель двух уже найденных placement findings.

placement_integrity_combo: необычное размещение подтверждается повреждением структуры

ПРИОРИТЕТ B · сильный или подтверждающий сигнал | family: unwind_integrity | severity: medium | weight: +8 дополнительно

Простыми словами	Metadata не только находится в подозрительном месте, но ещё и имеет ошибки чтения или внутренней структуры.
Что проверяет код	Есть unwind_exec_section или handler_nonexec_section и одновременно parse_fail, unwind_anomalies либо unwind_rva_oob.
Зачем это нужно	Отличить просто нестандартный layout от layout, сопровождаемого фактическими integrity problems.
Почему это важно	Две независимые стороны evidence взаимно усиливают вывод.
Нормальное объяснение	Новый unsupported format в нестандартной секции возможен, поэтому нужно проверить version/toolchain.
Как проверить вручную	Сначала подтвердить mapping, затем разобрать конкретную UNWIND_INFO и сравнить с prolog.
Как трактовать	Комбинационный бонус; особенно полезен для объяснения комиссии принципа corroboration.

writable_metadata_handler_combo: и metadata, и handler размещены в writable sections

ПРИОРИТЕТ B · сильный или подтверждающий сигнал | family: metadata_placement | severity: high | weight: +10 дополнительно

Простыми словами	Во время работы можно изменять и описание раскрутки, и код обработчика.
Что проверяет код	Одновременно unwind_writable_section > 0 и handler_writable_section > 0.
Зачем это нужно	Выделить целостную mutable exception-handling схему, а не единичное writable placement.
Почему это важно	Поведение обработки исключений может отличаться от статического файла после runtime modification.
Нормальное объяснение	JIT/runtime или специально управляемая code generation система.
Как проверить вручную	Запустить в VM, записать protections и bytes обеих областей, проверить вызовы function-table APIs.
Как трактовать	Сильный combined signal; не следует трактовать без учёта JIT/runtime context.

7. Приоритет C: контекстные и статистические эвристики

Эти patterns полезны для поиска unusual layouts, но имеют больше законных объяснений и требуют контекста.

unwind_reuse_strong: более 30% функций используют один UnwindInfoRVA

ПРИОРИТЕТ C · контекстная эвристика | family: unwind_reuse | severity: high | weight: +20	
Простыми словами	Сотни разных функций ссылаются на одно и то же описание раскрутки.
Что проверяет код	Только при n > 200 и max_count / n > 0.30.
Зачем это нужно	Найти необычно сильную централизацию metadata, характерную для dispatcher-style или heavily shared layouts.
Почему это важно	Множество функций может скрывать индивидуальные prologs за одной общей схемой либо использовать искусственно централизованную обработку.
Нормальное объяснение	Большое количество одинаковых tiny functions, compiler folding или runtime helper patterns.
Как проверить вручную	Выбрать несколько функций из группы, сравнить их реальные прологи и проверить, действительно ли одна запись им подходит.
Как трактовать	Высокая severity в модели, но по природе это статистический паттерн, а не формальная ошибка.

handler_dominance: один handler обслуживает более 80% записей с handlers

ПРИОРИТЕТ C · контекстная эвристика | family: handler_dominance | severity: medium | weight: +15	
Простыми словами	Почти все функции направляют исключения в один центральный обработчик.
Что проверяет код	handler_count > 50 и max_handler_count / handler_count > 0.80. Нормальный .text-like runtime handler без другого контекста подавляется.
Зачем это нужно	Найти central dispatcher/interceptor pattern, но не штрафовать обычный общий runtime helper.
Почему это важно	Центральный handler может скрывать реальную логику исключений или виртуализировать управление.
Нормальное объяснение	Языковая runtime-библиотека вполне может использовать общий handler. Поэтому код требует подтверждающего контекста.
Как проверить вручную	Открыть dominant handler в Ghidra, изучить callers/references и сравнить несколько функций, которые на него ссылаются.
Как трактовать	Нельзя использовать изолированно как доказательство. Реализация специально подавляет типичный benign case.

unwind_reuse_moderate: более 10% функций используют одну запись и есть другой сигнал

ПРИОРИТЕТ C · контекстная эвристика | family: unwind_reuse | severity: medium | weight: +10

Простыми словами	Заметная доля функций разделяет metadata, но finding появляется только при дополнительной проблеме.
Что проверяет код	n > 200, reuse ratio > 0.10, нет strong reuse и присутствует другой static finding.
Зачем это нужно	Не создавать false positive от обычного переиспользования, а учитывать его только как corroboration.
Почему это важно	В сочетании с placement/integrity/dominance это может указывать на централизованный нестандартный layout.
Нормальное объяснение	Само по себе умеренное sharing широко объяснимо оптимизациями.
Как проверить вручную	Сравнить shared group и дополнительный finding; отдельно reuse без контекста выводом не считать.
Как трактовать	Один из наиболее осторожных признаков: без другого сигнала полностью подавляется.

reuse_dominance_combo: reuse metadata совпадает с dominant handler

ПРИОРИТЕТ C · контекстная эвристика | family: unwind_reuse | severity: medium | weight: +8 дополнительно

Простыми словами	И описание раскрутки, и обработчик централизованы.
Что проверяет код	reuse ratio > 0.10 и уже существует handler_dominance.
Зачем это нужно	Выделить dispatcher-style систему, где несколько независимых связей сходятся к общим узлам.
Почему это важно	Паттерн может соответствовать виртуализатору, protector или общей runtime-машине.
Нормальное объяснение	Языковая runtime и однотипные функции также могут формировать такую картину.
Как проверить вручную	Построить граф functions → UnwindInfoRVA → HandlerRVA и исследовать центральные узлы.
Как трактовать	Статистический combined signal. Требует анализа кода центрального handler.

unwind_unusual_section: UNWIND_INFO находится не в xdata/rdata/pdata-like section

ПРИОРИТЕТ C · контекстная эвристика | family: metadata_placement | severity: medium при ≤5%, high при >5% | weight: +8 / +15

Простыми словами	Metadata лежит в секции с необычным именем или вообще не сопоставляется секции.
Что проверяет код	Section name не равен и не содержит xdata, pdata или rdata. Доля больше 5% усиливает finding.
Зачем это нужно	Подсказать аналитику, что layout отличается от распространённого compiler output.
Почему это важно	Может быть redirected/manual metadata, но имя секции само по себе ничего не доказывает.
Нормальное объяснение	Переименование секции, custom linker script, protector или другой toolchain.
Как проверить вручную	Смотреть не только имя, но permissions, содержимое, references и фактическую роль секции.
Как трактовать	Самый типичный кандидат на false positive. Для isolated placement-only случая confidence специально снижается.

pe_section_misaligned: секция нарушает заявленное выравнивание

ПРИОРИТЕТ C · контекстная эвристика | family: table_integrity | severity: medium | weight: +8

Простыми словами	VirtualAddress или PointerToRawData не кратны указанным alignment.
Что проверяет код	VirtualAddress % SectionAlignment != 0 либо raw data существует и RawPointer % FileAlignment != 0.
Зачем это нужно	Проверить согласованность section table с правилами размещения, объявленными самим PE.
Почему это важно	Может указывать на malformed layout и различия между интерпретацией loader и статических tools.
Нормальное объяснение	Редкие low-alignment images и нестандартные генераторы требуют отдельного изучения.
Как проверить вручную	Проверить значения FileAlignment/SectionAlignment и правила для конкретного типа image.
Как трактовать	Слабее OOB/overlap. Лучше использовать как дополнительный table-integrity signal.

no_runtime_functions: в executable image нет записей RUNTIME_FUNCTION

ПРИОРИТЕТ C · контекстная эвристика | family: table_integrity | severity: medium | weight: +10

Простыми словами	В файле есть исполняемый код, но exception table пуста.
Что проверяет код	n == 0 и присутствует непустая executable section. Для resource-only/nonexec image finding не создаётся.
Зачем это нужно	Обратить внимание на образ, где сложный nonleaf-код может остаться без unwind metadata.
Почему это важно	Раскрутка может быть невозможна для nonleaf functions, но leaf-only image может быть корректным.
Нормальное объяснение	Очень маленький leaf-only executable, ручной код без необходимости unwind или особая сборка.
Как проверить вручную	Дизассемблировать код и проверить, есть ли функции, изменяющие RSP или сохраняющие nonvolatile registers.
Как трактовать	Слабый standalone signal с несколькими mitigations. Не считать отсутствием metadata автоматически ошибкой.

8. Что скрывается внутри finding unwind_anomalies

В итоговой матрице все внутренние ошибки одной или нескольких UNWIND_INFO агрегируются в finding unwind_anomalies. Для понимания причины нужно смотреть bitmask UnwindSummary.anomalies. Ниже перечислены все 12 реализованных флагов.

1. UA_VERSION_UNKNOWN — Неизвестная версия UNWIND_INFO

Когда устанавливается	Version не равна 1 или 2.
Что это означает	Parser не может безопасно применять правила известных форматов и завершает разбор записи с parse failure.
Как проверить	Проверить, не является ли файл будущим V3/APX-образом или повреждённой записью.

2. UA_TRUNCATED — Структура обрезана

Когда устанавливается	Не хватает header, slots, operands, handler RVA или chained RUNTIME_FUNCTION.
Что это означает	Обязательные байты отсутствуют, поэтому значение следующих полей нельзя вывести надёжно.
Как проверить	Сверить file offset, CountOfCodes, выравнивание tail и фактический размер секции.

3. UA_BAD_OPCODE — Неизвестный или недопустимый unwind opcode

Когда устанавливается	Opcode не входит в поддерживаемый набор для указанной version либо ALLOC_LARGE имеет невозможный mode.
Что это означает	Операция не может быть интерпретирована по известной модели; parser продолжает её как один slot только для безопасной диагностики.
Как проверить	Сверить version и opcode с документацией и проверить, не сдвинут ли массив кодов.

4. UA_BAD_OPERAND — Противоречивый operand, flags или frame register

Когда устанавливается	Примеры: неизвестные flags; CHAININFO одновременно с EHANDLER/UHANDLER; FrameOffset без FrameRegister; RSP как сохраняемый register; повторный SET_FPREG; неправильный PUSH_MACHFRAME mode; frame mismatch в chain.
Что это означает	Opcode известен, но его параметры не соответствуют допустимой логике.
Как проверить	Разобрать конкретную operation и сравнить register/opinfo/flags с фактическим прологом.

5. UA_CODEOFFSET_OUTSIDE — CodeOffset выходит за SizeOfProlog

Когда устанавливается	Для обычной unwind operation CodeOffset больше заявленной длины пролога.
Что это означает	Операция утверждает, что действие пролога произошло после окончания самого пролога.
Как проверить	Сравнить SizeOfProlog, offsets operations и реальные адреса инструкций.

6. UA_CODEOFFSET_ORDER — Unwind operations имеют неправильный порядок CodeOffset

Когда устанавливается	Следующий CodeOffset больше предыдущего, хотя записи ожидаются в обратном порядке пролога.
Что это означает	Нарушение порядка может привести к неверной последовательности обратных действий.
Как проверить	Выписать offsets всех operations и сопоставить с прологом от конца к началу.

7. UA_OPERAND_SANITY — Значение формально читается, но выглядит нелогичным

Когда устанавливается	Примеры: overflow при сумме stack size; ALLOC_LARGE с нулевым/слишком маленьким размером; FAR save с нулевым, невыровненным или слишком малым offset.
Что это означает	Проверка ловит значения, противоречащие смыслу выбранной FAR/LARGE encoding.
Как проверить	Проверить единицы измерения и реальную stack allocation; этот bit является sanity-эвристикой.

8. UA_ALIGNMENT — UnwindInfoRVA не выровнен на 4 байта

Когда устанавливается	unwind_rva & 3 не равно нулю.
Что это означает	UNWIND_INFO и её tail ожидают корректное выравнивание; misalignment может означать неправильную ссылку.
Как проверить	Проверить RVA, section start и наличие padding.

9. UA_HANDLER_OOB — HandlerRVA выходит за SizeOfImage

Когда устанавливается	HandlerRVA >= SizeOfImage.
Что это означает	Адрес обработчика не может принадлежать текущему image.
Как проверить	Проверить tail offset и сам RVA; затем исключить ошибочное чтение из-за неверного CountOfCodes.

10. UA_CHAIN_INVALID — Chained RUNTIME_FUNCTION имеет недопустимые поля	
Когда устанавливается	End <= Begin, диапазон вне image, UnwindRVA нулевой или вне image.
Что это означает	Связанная запись не может корректно описывать функцию и следующую UNWIND_INFO.
Как проверить	Разобрать 12-байтный chain tail и проверить каждый RVA отдельно.

11. UA_CHAIN_LOOP — Обнаружен цикл chained unwind	
Когда устанавливается	В цепочке повторно встречается уже посещённый UnwindInfoRVA.
Что это означает	Без защиты traversal зациклится, а metadata не имеет конечной основы.
Как проверить	Построить граф chain links и показать повторяющийся узел.

12. UA_CHAIN_DEPTH_EXCEEDED — Цепочка длиннее разрешённого предела	
Когда устанавливается	Traversal достиг max_chain_depth, по умолчанию 8, CLI допускает 1–16.
Что это означает	Ограничение защищает от чрезмерной рекурсии и maliciously deep metadata.
Как проверить	Повторить с обоснованно большим limit до 16 и изучить, заканчивается ли цепь.

9. Пороги и почему они нужны

Порог	Где используется	Смысл
>1%	invalid_ranges, rf_oob, unwind_rva_oob, parse_fail	Отделяет единичный дефект от широкой integrity-проблемы.
>5%	unwind_anomalies, unusual/writable placement	Показывает, что pattern охватывает заметную часть таблицы.
>10% и n>200	moderate unwind reuse	Рассматривается только как кандидат и требует другого finding.
>30% и n>200	strong unwind reuse	Очень сильная централизация одной UNWIND_INFO.
>80% и >50 handlers	handler dominance	Почти все handlers ведут к одной цели; benign runtime case подавляется.
<32 functions	small_sample_size mitigation	Маленький sample делает проценты нестабильными.

Честная трактовка порогов

Это консервативные engineering defaults. Они делают правила стабильными и объяснимыми, но не являются статистически оптимальными для всех компиляторов, runtime и protectors. Проект не заявляет ROC-calibration или машинное обучение.

10. Score, suspicion, severity и confidence

Score

Каждый finding добавляет фиксированный вес. Сумма насыщается на 100. Score отвечает на вопрос: насколько высоко этот файл нужно поставить в очередь ручного анализа. Он не отвечает на вопрос о вероятности атаки.

Suspicion

Это score / 100. Поле удобно для API, но не превращает engineering score в статистическую вероятность.

Severity

Severity принадлежит конкретному finding: medium или high. Она описывает силу нарушения в матрице.

Evidence level

Уровень	Правило
strong	Не менее двух strong findings либо score ≥ 70.
medium	Не менее одного strong, двух medium либо score ≥ 30.
weak	Есть finding/score, но условия выше не выполнены.
none	Нет findings и score = 0.

Confidence

Confidence начинается с 0.15, увеличивается за независимые evidence families, strong/medium findings и hard integrity problems, затем уменьшается mitigations. Результат ограничивается диапазоном 0.05–0.90. При нулевом score используется baseline 0.20.

Компонент	Изменение
Каждая evidence family	+0.11
Каждый strong finding	+0.08
Каждый medium finding	+0.04
Hard table-integrity	+0.08
Hard unwind-integrity	+0.08

11. Ослабляющие правила: защита от слишком сильного вывода

Mitigation не удаляет finding и не уменьшает score. Оно уменьшает confidence и сообщает аналитику, почему у сигнала остаётся правдоподобное benign explanation.

single_signal_only	-0.12 confidence
Когда	Сработал ровно один несильный finding.
Зачем	Один контекстный симптом слишком легко имеет нормальное объяснение. Score остаётся, но уверенность в интерпретации снижается.

small_sample_size	-0.06 confidence
Когда	Меньше 32 функций, findings есть, но strong finding отсутствует.
Зачем	На маленькой таблице один случай сильно искажает проценты и ratios. Нужна осторожность.

placement_only_signal	-0.10 confidence
Когда	Есть только unusual section без execute/write, handler, range или parse problems.
Зачем	Необычное имя секции часто объясняется toolchain или linker script и не должно давать сильный вывод.

single_family_only	-0.08 confidence
Когда	Нет strong finding, а все сигналы принадлежат одной evidence family.
Зачем	Несколько однотипных симптомов слабее независимого подтверждения из другой части модели.

Пример

Одна UNWIND_INFO находится в read-only секции .weird, а остальные признаки отсутствуют. Finding unusual_section сохраняется, но single_signal_only, small_sample_size, placement_only_signal и single_family_only снижают confidence. Правильный вывод: проверить custom linker layout, а не объявлять файл вредоносным.

12. Verdict и обязательная ручная проверка

Verdict	Score	Смысл
`no_strong_static_anomalies`	0–14	Сильных static findings нет. Это не доказательство полной безопасности.
`low_suspicion`	15–39	Есть признаки, которые нужно проверить, но evidence ограничено.
`moderate_suspicion`	40–69	Несколько или достаточно сильные признаки.
`high_suspicion`	70–100	Сильная совокупность findings; приоритетный ручной анализ.

Когда requires_manual_review = true

- score ≥ 20;
- metadata находится в executable или writable section;
- handler находится в non-executable/unknown или writable section;
- есть UNWIND_INFO parse failures;
- применено хотя бы одно mitigation.

Почему mitigation тоже требует ручной проверки

Mitigation означает, что finding есть, но его автоматическая интерпретация ослаблена. Именно в таком случае решение должен принять эксперт после просмотра секций, кода и runtime context.

13. Динамические признаки и их сила

Dynamic evidence не заменяет static findings. Оно отвечает на другой вопрос: что реально произошло в конкретном запуске под Windows debugger.

Сигнал	Сила в runtime model	Почему
First-chance exception	weak	Исключение возникло, но может быть штатно обработано.
VEH/VCH/UEF registration	weak	Handler зарегистрирован, но ещё не доказано, что он использовался.
Second-chance exception	medium	Обычная обработка не справилась; событие требует внимания.
RtlAddFunctionTable / callback	medium	Появляется runtime unwind metadata, отсутствующая в статической таблице.
Unwind handler / VEH / VCH / UEF hit	strong component	Подтверждено фактическое выполнение handler path.
Handler hit + medium signal	strong runtime level	Факт выполнения подтверждается дополнительным runtime context.

Когда считается static/dynamic correlation

- Есть подозрительный static handler context и во время запуска произошёл handler/VEH/VCH/UEF hit.
- Есть static placement anomaly и наблюдался RtlAddFunctionTable или RtlInstallFunctionTableCallback.

Граница корреляции

Это coarse correlation между классами признаков. Она не доказывает, что конкретная static запись вызвала конкретную runtime инструкцию. Для такого доказательства потребовался бы per-function event graph.

14. Итоговая case strength

Класс	Правило	Как понимать
`strong_correlated_case`	Есть correlation и static либо runtime strong.	Самая сильная совокупность в текущей модели.
`medium_correlated_case`	Есть correlation и одна сторона medium.	Связь подтверждена, но evidence не максимальное.
`strong_unconfirmed_case`	Strong evidence без cross-layer correlation.	Сильный сигнал, который ещё не подтверждён другим слоем.
`medium_case`	Static или runtime medium.	Нужен ручной анализ и желательно подтверждение.
`weak_case`	Только weak evidence.	Направление исследования, не вывод.
`no_case`	Нет meaningful evidence.	В рамках реализованной модели case не построен.

При наличии любых dynamic signals итоговая confidence увеличивается на 0.10; при correlation добавляется ещё 0.10. Итог ограничивается 0.98. Это также engineering confidence, а не вероятность.

15. Как читать реальный отчёт: пошаговый алгоритм

Шаг	Действие	Зачем
1	**Проверить parse status**	Если PE или exception table не разобраны, сначала работать с parse error, а не со score.
2	**Посмотреть strong structural findings**	OOB, overlaps, invalid ranges и parse failures имеют приоритет над необычными именами секций.
3	**Раскрыть unwind_anomalies**	Найти конкретные UA_* bits и parse_error для representative RVA.
4	**Проверить affected_count и долю**	Одна запись и массовый pattern требуют разной трактовки.
5	**Посмотреть evidence families**	Независимые families повышают достоверность case.
6	**Прочитать mitigations**	Они перечисляют причины, почему автоматический вывод должен быть ослаблен.
7	**Проверить location**	Открыть representative RVA в PE viewer, hex editor и Ghidra.
8	**Сопоставить с динамикой**	Handler hits и runtime tables могут подтвердить static hypothesis.
9	**Выполнить manual_validation_steps**	Отчёт уже формирует список конкретных действий.
10	**Сформулировать ограниченный вывод**	Говорить о нарушенной структуре или подозрительном паттерне, а не автоматически о malware.

16. Пять типовых сценариев интерпретации

Чистый compiler-generated image	
Наблюдение	score = 0, findings отсутствуют, evidence_level = none.
Правильный вывод	Инструмент не нашёл сильных аномалий в реализованной модели. Это не означает, что весь код безопасен.

Одна read-only секция с необычным именем	
Наблюдение	unwind_unusual_section, низкий score, несколько mitigations.
Правильный вывод	Сначала проверить custom linker/toolchain. Это слабый placement-only case.

Пересечения и OOB в .pdata	
Наблюдение	rf_overlaps + rf_oob + возможно unwind_rva_oob.
Правильный вывод	Сильное table-integrity повреждение. Дальнейшие статистические эвристики вторичны.

Writable metadata и writable handler	
Наблюдение	два placement findings + writable_metadata_handler_combo.
Правильный вывод	Проверить JIT/runtime explanation и изменения памяти динамически.

Dominant handler с runtime hit	
Наблюдение	handler_dominance + фактический handler/VEH/VCH/UEF hit.
Правильный вывод	Есть correlated case. Нужно разобрать central dispatcher и подтвердить его назначение.

17. Что система признаков утверждает и чего не утверждает

Можно утверждать	Нельзя утверждать
Нарушен конкретный структурный инвариант.	Файл обязательно вредоносный.
Metadata имеет необычное размещение или reuse pattern.	Обнаружен конкретный protector без отдельной сигнатуры.
Результат детерминирован по опубликованной матрице.	Score является статистической вероятностью.
Runtime handler/API был замечен в данном запуске.	Исследованы все возможные execution paths.
Benign corpus прошёл без findings в проверенной ревизии.	Ложные срабатывания невозможны на всех программах.
Findings локализованы и проверяемы вручную.	Эксперт больше не нужен.

Формулировка для защиты

Признаки выбраны в два этапа: сначала формальные инварианты PE/COFF и Windows x64 unwind, затем интерпретируемые placement и statistical heuristics. Сильные структурные нарушения имеют больший приоритет, слабые признаки ослабляются mitigations и требуют corroboration. Система предназначена для triage и ручной верификации, а не для автоматического доказательства вредоносности.

18. Краткая памятка по важности

Сначала	Затем	В последнюю очередь
OOB, overlaps, invalid ranges, parse failures, unknown/truncated UNWIND_INFO.	Executable/writable metadata, nonexec/writable handlers, combined findings.	Unusual names, reuse, dominance, misalignment и empty table.
Они ставят под сомнение саму структуру.	Они показывают сильное несоответствие роли и placement.	Они чаще имеют benign explanations.

Файлы реализации, с которыми согласован документ

- src/heuristics.c — findings, weights, thresholds, confidence и mitigations;
- src/unwind.c и include/unwind.h — 12 внутренних UA_* anomalies;
- src/pe.c — section и exception-directory validation;
- src/report.c — runtime evidence, correlation, case strength и manual steps;
- docs/evidence_matrix.md — зафиксированная матрица доказательств;
- docs/threshold_calibration.md — честные границы калибровки.

Главная идея системы: сначала установить, можно ли доверять структуре metadata; затем определить, насколько необычно она организована; после этого найти независимое статическое или динамическое подтверждение и только тогда формулировать ограниченный экспертный вывод.